

SIMATS ENGINEERING

ASSESSMENT TOOL 2 – INDUSTRY PROBLEM SOLVING TASK

Cloud Computing and Big Data Analytics (CSA1558)

CO4: Analyze big data characteristics and challenges across domains including security and application aspects.

ALL 5 QUESTIONS – DETAILED ANSWERS | Total: 40 Marks

The answers below follow the five industry scenarios in the uploaded question paper and provide a practical, scalable solution with tools, analytics, security/risk considerations, and implementation reasoning.

1. Retail Data Optimization Problem

Problem understanding: The retail chain operates physical stores and an online platform. It generates large volumes of sales, inventory and customer-behavior data. The required outcomes are real-time inventory tracking, demand forecasting and personalized marketing recommendations.

Proposed architecture:

1. Data sources: Collect point-of-sale transactions, online orders, product views, carts, inventory updates, customer interactions and supplier information. Store structured transactional information in operational databases while sending high-volume events to a distributed streaming layer.

2. Data ingestion: Use Apache Kafka as a distributed event-streaming platform. Store events in separate topics such as sales, inventory, customer activity and product events. Kafka partitions allow many consumers to process data in parallel.

3. Data lake: Store raw historical and semi-structured data in HDFS or cloud object storage. Organize data by date, store, region and event type. Keep raw data separately from cleaned and curated datasets so that transformations can be reproduced.

4. Real-time inventory tracking: Stream inventory and sales events through Kafka to a stream-processing engine such as Apache Flink or Spark Structured Streaming. The processor can update current stock levels and detect low-stock or out-of-stock conditions. A fast NoSQL store or cache can serve current inventory to applications.

5. Demand forecasting: Historical sales, seasonality, promotions, holidays, prices, weather and regional trends can be transformed into features. Time-series and machine-learning models such as ARIMA/Prophet-style forecasting or gradient-boosting/regression models can estimate future demand. More advanced models can be used if the dataset and business requirements justify them.

6. Personalized marketing: Customer browsing, purchases, product preferences and response to previous campaigns can be used to build recommendation features. Collaborative filtering, content-based recommendation or hybrid recommendation models can generate personalized products/offers.

7. Batch analytics: Apache Spark or Hadoop-based processing can analyze historical sales, customer segmentation and product performance. Aggregated datasets can be stored in an analytical warehouse or queried through engines such as Spark SQL/Trino.

8. Dashboard layer: BI tools can display real-time inventory, sales trends, forecasted demand, customer segments and campaign performance. Operational dashboards can consume streaming aggregates, while management reports can use historical analytical tables.

9. Scalability: Use horizontal scaling: add Kafka brokers, processing workers and storage capacity as the workload increases. Partition data by suitable keys and avoid creating hot partitions.

10. Security: Apply authentication, authorization, encryption, audit logging and least-privilege access. Customer information should be protected according to applicable privacy requirements.

Result: The architecture combines streaming for immediate inventory visibility with batch and ML analytics for forecasting and personalization, allowing the retail company to react quickly while also making long-term decisions from historical data.

2. Social Media Fraud Detection Problem

Problem understanding: The platform processes billions of posts, comments, likes and shares every day and must detect fake accounts and misinformation quickly. This requires high-throughput ingestion, near-real-time processing and machine-learning-based classification.

Proposed architecture:

1. Event ingestion: Web/mobile services publish user interactions to Apache Kafka or another distributed event platform. Topics can separate posts, comments, reactions, account events and reports. Partitioning distributes billions of events across brokers.

2. Stream processing: Use Apache Flink, Spark Structured Streaming or Kafka Streams to process events continuously. The stream processor can calculate features such as posting frequency, account age, interaction patterns, sudden changes in behavior, sharing velocity and network relationships.

3. Fake-account detection: A machine-learning model can use account metadata and behavioral features to estimate the probability that an account is fraudulent. Models may include logistic regression, random forests, gradient boosting or neural-network approaches depending on the available data and accuracy requirements.

4. Misinformation detection: Text, metadata, propagation patterns and source information can be analyzed. Natural-language-processing models can classify content, while graph/behavioral features can identify unusual propagation patterns. Human review can be included for high-impact or uncertain cases.

5. Real-time decision layer: A low-latency feature store/cache can hold recent user and content features. New events are scored immediately. High-risk events can trigger moderation queues, additional verification or other policy-defined actions.

6. Historical training data: Store historical events in a data lake. Batch processing with Spark can clean and label training datasets and calculate long-term behavioral features. Models can be retrained periodically as fraud patterns change.

7. Scalability and fault tolerance: Kafka replication protects events from broker failure. Stream processors can run parallel tasks and recover using checkpoints/state. Distributed storage provides scalable retention of historical interactions.

8. Monitoring: Track event-processing latency, throughput, model precision/recall, false positives, false negatives and processing failures. Model performance should be monitored because attackers can change behavior over time.

9. Security and privacy: Apply access controls, encryption, audit logging and data-minimization policies. Sensitive user information should not be exposed unnecessarily to analytics components.

Result: The combination of Kafka-based ingestion, stream processing and ML provides near-real-time fraud and misinformation detection while the data lake and batch processing support model training and long-term analysis.

3. Government Data Platform Problem

Problem understanding: The government agency combines census, economic and geospatial datasets from multiple departments. The major requirements are shared access, interoperability, strict privacy and security.

Proposed governance and data-management strategy:

- 1. Central data platform:** Use a governed data lake/data warehouse architecture. A raw zone stores source data, a standardized zone contains cleaned and normalized data, and a curated zone provides approved datasets for analytics and department-level applications.
 - 2. Interoperability:** Establish common data definitions, schemas, identifiers and metadata standards. Departments should use agreed formats and APIs for exchanging information. Data integration/ETL tools can transform department-specific formats into common representations.
 - 3. Metadata catalog:** Maintain a catalog describing dataset ownership, source, schema, update frequency, quality, sensitivity classification and permitted usage. Data lineage should show where a dataset came from and how it was transformed.
 - 4. Data quality:** Apply validation rules for missing values, duplicate records, inconsistent identifiers, invalid geographic coordinates and conflicting records. Quality checks should run during ingestion and transformation.
 - 5. Access control:** Use centralized identity management and role-based or attribute-based authorization. A department should receive only the datasets and fields necessary for its approved purpose. Apply least privilege.
 - 6. Privacy:** Classify data according to sensitivity. Personally identifiable or confidential information should receive stronger controls. Where possible, analytical datasets can use masking, pseudonymization or controlled de-identification.
 - 7. Encryption and security:** Encrypt data during transmission and at rest. Protect APIs and ingestion endpoints with authentication and authorization. Use network segmentation and monitoring for sensitive systems.
 - 8. Audit and compliance:** Record access, modification, sharing and administrative actions. Regular audits should identify unusual access patterns and policy violations. Retention and deletion rules should be enforced according to applicable government policies.
 - 9. Availability and recovery:** Use redundant storage, backups, replication and tested disaster-recovery procedures. Critical government services should not depend on a single server or storage location.
 - 10. Shared analytics:** Approved datasets can be exposed through SQL/query engines, dashboards and analytical services. Geospatial datasets can support maps and location-based analysis, while census/economic data can support policy planning.
- Result:** A governed data lake with standardized metadata, interoperability controls, strong security and controlled departmental access provides a common national platform without sacrificing privacy or accountability.

4. Banking Fraud Detection Problem

Problem understanding: The bank processes millions of transactions every minute. Fraud detection must combine historical and streaming data while meeting very low latency and high accuracy requirements.

Proposed big data pipeline:

- 1. Transaction ingestion:** Each transaction event is published to a distributed messaging platform such as Kafka. Events should contain transaction ID, customer/account identifier, amount, timestamp, merchant information, location and relevant risk attributes.
- 2. Stream processing:** Apache Flink or Spark Structured Streaming can process transactions continuously. The system can calculate features such as transaction velocity, amount deviation from normal behavior, unusual location, rapid transactions from different regions and merchant risk.
- 3. Real-time feature store:** A low-latency database/cache can maintain recent customer behavior, account balances where appropriate, historical risk indicators and model features. The scoring service retrieves these features quickly.
- 4. Machine-learning detection:** Supervised models such as logistic regression, random forests or gradient boosting can classify transactions when labeled fraud data is available. Anomaly-detection models can identify unusual transactions when labels are incomplete. A hybrid system can combine rules and ML scores.

5. Rules engine: Deterministic rules can handle obvious high-risk conditions, such as impossible transaction patterns or policy-defined limits. Rules and ML can be combined so that the system is both explainable and adaptive.

6. Historical analytics: Store historical transaction data in a distributed data lake or warehouse. Spark can calculate long-term customer behavior, train models, analyze fraud patterns and evaluate false-positive rates.

7. Decision and alerting: The real-time scoring layer can classify transactions as low, medium or high risk according to the bank's policies. High-risk cases can trigger additional verification or investigation workflows. The exact action should follow the bank's fraud and compliance policies.

8. Accuracy: Continuously monitor precision, recall, false positives and false negatives. Models should be retrained as fraud patterns evolve. Human investigation outcomes can become new training labels.

9. High availability: Kafka replication, redundant stream-processing instances, replicated storage and failover mechanisms help keep fraud detection available. Checkpointing allows stream processing to recover state after failures.

10. Security: Encrypt financial data, apply strict access controls, audit data access and separate sensitive transaction information from general analytics where required.

Result: The combination of streaming, low-latency feature retrieval, rules and ML enables rapid transaction scoring, while the historical platform supports model training and continuous improvement.

5. Agriculture Smart Farming Problem

Problem understanding: The smart-farming system collects soil, weather and crop data from distributed sensors and drones. Farmers need actionable insights for improving yield and reducing unnecessary resource use.

Proposed precision-agriculture architecture:

- 1. Data sources:** Collect soil moisture, temperature, nutrient measurements, weather conditions, crop images, drone observations and farm activity data. Each event should include a timestamp and location/field identifier.
 - 2. Edge and IoT ingestion:** Sensors can send data through an IoT gateway. Edge processing can filter invalid readings, aggregate frequent measurements and temporarily buffer data when network connectivity is poor.
 - 3. Distributed messaging:** A messaging system such as Kafka or an IoT-compatible event platform can receive sensor streams. Partitioning by farm, field or sensor group allows parallel processing.
 - 4. Storage:** Use a data lake/object store or HDFS for raw sensor records, drone imagery and historical datasets. A time-series database can support fast queries of recent sensor measurements. Metadata should associate observations with farms, fields and geographic coordinates.
 - 5. Real-time analytics:** Stream processing can detect low soil moisture, extreme temperature, abnormal sensor readings or weather-related risks. Alerts can be sent to farmers or farm-management applications.
 - 6. Crop and yield prediction:** Historical weather, soil, crop and yield information can train machine-learning models. Regression or gradient-boosting models can estimate expected yield. Image-based models can analyze drone imagery for crop stress or disease indicators.
 - 7. Resource optimization:** Analytics can recommend irrigation timing, fertilizer usage or other farm actions based on soil conditions, crop requirements and weather forecasts. The objective is to use resources efficiently while protecting crop health.
 - 8. Batch analytics:** Historical farm data can be processed using Spark or another distributed analytics framework. This can reveal seasonal patterns, compare fields, evaluate interventions and improve future models.
 - 9. Geospatial analysis:** Field boundaries, sensor coordinates and drone imagery can be combined to create field-level maps. Farmers can identify areas requiring different treatment rather than applying the same resource level across an entire farm.
 - 10. Decision support:** Dashboards can display soil moisture, weather, crop health, predicted yield, irrigation recommendations and alerts. The system should present understandable recommendations so farmers can make informed decisions.
 - 11. Reliability and security:** Sensor data should be authenticated where possible, communication should be protected, and critical data should be backed up. Edge buffering helps prevent data loss during temporary connectivity failures.
- Result:** The architecture combines IoT/edge ingestion, distributed storage, real-time streaming, geospatial analytics, historical processing and ML. This allows farmers to move from periodic manual observation to data-driven precision agriculture and better resource management.

Exam conclusion: Across all five industry problems, the common design principles are distributed ingestion, scalable storage, parallel/stream processing, analytics or machine learning, appropriate security controls, fault tolerance and a clear dashboard/decision-support layer.